

Executive Summary

Project: Migration & Stabilization — Setting Up & Migrating a Windows Server and Client

Author: Amanda Kondrat'yev · Per Scholas (Houston) · AI-Enabled IT Support Capstone

Repository: <https://github.com/manderwall/it-support-capstone>

Overview

This project simulates a small enterprise IT environment and the kind of disruption a support specialist is expected to handle: build a stable Windows network, put it through a required migration that intentionally causes problems, and then diagnose the fallout methodically rather than by guesswork. The entire environment was built from scratch in isolated virtual machines, every step was automated with reviewed scripts, and all work was captured as evidence — transcripts and screenshots — so the results can be independently verified.

What was built

A self-contained Windows domain running on two isolated virtual machines:

- **A Windows Server 2022 domain controller** providing centralized login (Active Directory), name resolution (DNS), and a shared network folder secured to an authorized group with least-privilege permissions.
- **A Windows 11 client** joined to that domain, so a single set of credentials controls access across the network.

The build was provisioned with PowerShell automation — reviewed line by line before it was run — and then independently validated: the instructor's verification script returned **PASS on every check**.

The migration and its impact

With safe rollback snapshots in place, the provided migration was run once on the server, exactly as issued, and its completion recorded. By design, it left a trail of six user-facing incidents — from “access denied” on the shared folder to a missing printer and slow logins.

What the investigation found

Each of the six incidents was investigated on the affected machine using read-only diagnostics. A clear pattern emerged: **the migration changed access control and policy, not the core network**. Login, DNS, domain trust, and time synchronization all remained healthy, which ruled out a network or trust failure and pointed directly to the real causes:

- The shared folder's file-level permissions had been stripped, blocking access (the root cause behind two of the tickets).
- A new account policy was introduced by the migration, accounting for the slow logins and settings problems.
- The printer had been removed, and several background services were left stopped.

Every diagnosis is backed by captured evidence, and each one identifies precisely what a corrective fix would need to target.

Status and next steps

The submission delivers fully completed, evidence-backed work through the investigation stage (Phases 1–3): the verified baseline build, the executed migration, and the complete incident diagnosis. The remaining work — applying the corrective fixes (Phase 4) — is well-scoped by the diagnoses above and can be resumed at any time from the preserved known-good snapshots. The decision to conclude here was driven by project timeline and repeated upstream changes to the migration script, not by any technical blocker.

What this demonstrates

Beyond the technical build, the project reflects how infrastructure work *should* be done: isolate the environment, snapshot before every change, let evidence define the problem, make the smallest correct fix, and verify it. That same discipline governed the use of AI throughout — treated as a fast drafting assistant whose every output was read, understood, and verified before it was trusted, never as an authority. A documented example: an AI-influenced DNS setting failed verification during the build, was diagnosed to a single misconfigured address, corrected with one targeted change, and re-verified to a full pass.